

Cost Optimization in Cloud Computing: A Forecast-Driven System for Capacity Reservation under Intermittent Random Demand Surges

First Author, Second Author, Third Author, Fourth Author

Faculty of Computer and Information Sciences

Ain Shams University

Cairo, Egypt

{author1, author2}@cis.asu.edu.eg

Supervisor Name

Faculty of Computer and Information Sciences

Ain Shams University

Cairo, Egypt

supervisor@cis.asu.edu.eg

Abstract—Enterprise cloud expenditure continues to grow, and a large share of it is wasted because provisioned capacity is poorly matched to a demand that combines a roughly stationary *base* with *intermittent random surges* driven by planned or unplanned events. A firm meets this demand through three pricing instruments: cheap but rigid *base contracts* (long-term reserved instances), flexible *supplementary contracts* (short or cancellable reserved instances), and a punitive *on-demand spillover* for any uncovered load. Classical operations-research theory shows that the optimal reservation level follows a newsvendor critical-fractile rule and that the supplementary-capacity decision obeys a two-threshold (s, S) policy. This paper operationalizes that theory as a deployable, data-driven system, Smart Cloud Optimizer, which (i) collects cost and utilization telemetry from Amazon Web Services, (ii) estimates the demand distribution with an anomaly-aware ensemble of five forecasting models, (iii) sets reserved capacity from the estimated critical fractile and sizes surge headroom at the 95th percentile of load, and (iv) realizes the threshold policy through a mixed-integer program and eight commitment rules. In a numerical study on a twelve-month workload built from open production traces, the forecasting ensemble attains a mean absolute percentage error of 7.9% at a one-week horizon, and the optimizer recovers \$590.40 of monthly cost ($\approx 26\%$ of the bill) across 19 recommendations. The study confirms that pairing distribution estimation with newsvendor reservation and threshold-based supplementary capacity yields substantial, automated, and explainable savings.

Index Terms—cloud computing, cost optimization, capacity reservation, reserved instances, demand surges, newsvendor model, threshold policy, demand forecasting, FinOps

I. INTRODUCTION

Worldwide public-cloud spending is projected to approach \$600 billion annually [1], and industry surveys consistently report that roughly a quarter to a third of it is wasted on idle or mispriced resources [2]. The root cause is a mismatch between *provisioned* capacity and *realized* demand. This mismatch is most costly when demand is not stationary: a steady *base* load is overlaid with *intermittent random surges*—marketing campaigns, product launches, viral content, or batch jobs—whose timing and magnitude are uncertain.

An enterprise cloud user (the *firm*) manages this demand through three purchasing instruments [3], [4]:

- **Base contracts:** reserved instances on the standard long contract (typically one or three years), priced at a deep discount to on-demand, used to cover the stationary base demand.
- **Supplementary contracts:** additional reserved instances, of standard or shorter length, deployed to cover surges; they may be cancelled for a fee or allowed to expire.
- **On-demand spillover:** any demand left uncovered spills onto pay-as-you-go pricing—the flexible but punitive option.

The firm’s problem is to choose the base capacity and a policy for purchasing, renewing, and cancelling supplementary capacity so as to minimize expected total cost. Chen, Lei, and Moinszadeh [3] formalize this problem and prove two structural results that organize the entire solution space: the optimal capacity follows a *newsvendor-type* critical fractile, and the optimal supplementary-capacity policy is a *two-threshold* (s, S) policy borrowed from inventory theory.

That analysis assumes the demand distribution is given in closed form. In practice, a firm does not know its surge distribution *a priori*; it must estimate it from telemetry, and the estimate must drive the reservation decision automatically across a heterogeneous fleet. This paper takes that *systems-and-data* step. We present Smart Cloud Optimizer, a deployed platform that turns raw Amazon Web Services (AWS) billing and CloudWatch data into ranked, explainable reservation and right-sizing actions, using the newsvendor and threshold structure of [3] as its decision backbone.

Contributions.

- We cast a working cloud-cost system in the base/supplementary reservation framework of [3], mapping reserved instances to base contracts, surge headroom to supplementary capacity, and pay-as-you-go to on-demand spillover (Sections III–IV).
- We replace the assumed demand distribution with a learned one: an anomaly-aware ensemble of five forecasting models estimates the base demand and the surge tail that feed the newsvendor critical fractile (Section III).

- We realize the (s, S) supplementary-capacity policy with a mixed-integer program for right-sizing and eight commitment rules, sizing surge headroom at the 95th percentile of load (Section IV).
- We report a numerical study on a twelve-month workload built from open production traces, measuring forecasting accuracy and \$590.40/month ($\approx 26\%$) of recoverable cost (Section V).

The remainder of the paper follows the structure of the anchor work: Section II reviews the literature, Section III states the model and the newsvendor reservation result, Section IV develops the threshold policy and its data-driven realization, Section V presents the numerical study, and Section VI concludes.

II. LITERATURE REVIEW

The reserved-versus-on-demand decision under demand uncertainty has been attacked from three largely complementary directions.

A. Stochastic and Robust Programming

The canonical formulation is the broker’s two-stage stochastic integer program of Chaisiri *et al.* [5]: stage one commits to reservations under demand and price uncertainty, stage two buys on-demand to cover the realized shortfall, solved by sample-average approximation. Multi-stage extensions model the rolling-horizon nature of reservations [6], and robust optimization with a budget of uncertainty [7] is the natural alternative when the distribution itself is unknown. These models scale to many instance types but require the demand distribution as input and yield no closed-form policy.

B. Inventory- and Queueing-Theoretic Models

The anchor paper [3] sits in this tradition: its newsvendor and (s, S) results are direct adaptations of inventory theory [8]. Furman and Diamant [9] solve the provider-side dual, optimal capacity planning under periodic time-varying demand. The queueing strand models bursty arrivals with Markov-modulated Poisson processes that fit real cloud traces well [10], producing queue-threshold activation policies directly analogous to supplementary-capacity activation. Our work is data-driven but keeps this theory as its skeleton.

C. Learning-Based Methods

A simpler pipeline forecasts demand and feeds the forecast to a reservation optimizer. Strong workload predictors include ARIMA [11], Prophet [12], Holt-Winters/ETS [13], and transformer models [14]; large-provider studies predict VM behavior at datacenter scale [15]. Reinforcement learning has been applied widely to autoscaling [16]–[18], though the long lease horizon of reservations complicates credit assignment. We adopt the forecast-then-optimize pattern but, unlike pure autoscaling, target the cost signal and feed it into the newsvendor reservation rule. The emerging FinOps discipline [19] frames the practical need our system addresses.

III. SYSTEM MODEL AND NEWSVENDOR RESERVATION

A. Architecture

Smart Cloud Optimizer follows a three-stage pipeline—*collect*, *analyze*, *optimize*—over a single multi-tenant store (Fig. 1). A collector assumes a cross-account IAM role via STS and pulls twelve months of cost, utilization, inventory, and pricing data from AWS Cost Explorer, CloudWatch, and the Pricing API for ten services; only the role ARN is persisted, never long-lived secrets. Telemetry is normalized into 30 relational tables keyed by `(timestamp, user_id, resource_id)`; the *analyze* and *optimize* stages read from this store, and a dashboard presents ranked recommendations.

B. Demand Model

Let d_t denote realized demand (here, daily cost) on day t . Following the base/surge decomposition of [3], we write

$$d_t = \underbrace{b_t}_{\text{base}} + \underbrace{s_t}_{\text{seasonal}} + \underbrace{u_t}_{\text{surge}} + \varepsilon_t, \quad (1)$$

where b_t is a slowly varying base level, s_t a deterministic weekly seasonal term, $u_t \geq 0$ an intermittent surge that is zero on most days, and ε_t noise. The base and seasonal parts are predictable and are served by base contracts; the surge part is, by construction, hard to predict and is served by supplementary capacity held as headroom. Unlike the anchor model, the distribution of d_t is *not* given: it is estimated online by the forecasting ensemble of Section IV-A.

C. Newsvendor Reservation

Consider a unit of capacity over a commitment period. Let c_r be the reserved unit price and $c_o > c_r$ the on-demand unit price. Reserving one unit too many wastes the reserved cost $c_o^+ = c_r$ (overage); reserving one unit too few forces on-demand spillover at premium $c_u = c_o - c_r$ (underage). The classical newsvendor solution sets the reserved capacity q^* at the critical fractile of the demand distribution F :

$$F(q^*) = \frac{c_u}{c_u + c_o^+} = \frac{c_o - c_r}{c_o} = 1 - \frac{c_r}{c_o}, \quad q^* = F^{-1}\left(1 - \frac{c_r}{c_o}\right). \quad (2)$$

The deeper the reservation discount $1 - c_r/c_o$, the higher the fractile and the more capacity the firm should commit—exactly the structural insight of [3]. Equation (2) is the bridge between the theory and our system: the forecasting stage estimates F , and the optimization stage commits capacity at the implied quantile. With the $\approx 37\%$ EC2 reservation discount observed in our data ($c_r/c_o \approx 0.63$), the base contract is set near the 37th percentile of demand and the residual surge mass is covered by supplementary headroom, sized as in Section IV-B. Table I summarizes the notation.

IV. SUPPLEMENTARY-CAPACITY POLICY AND ITS REALIZATION

The anchor paper proves that the optimal management of supplementary contracts is a two-threshold (s, S) policy: when active supplementary capacity falls below s , purchase up to

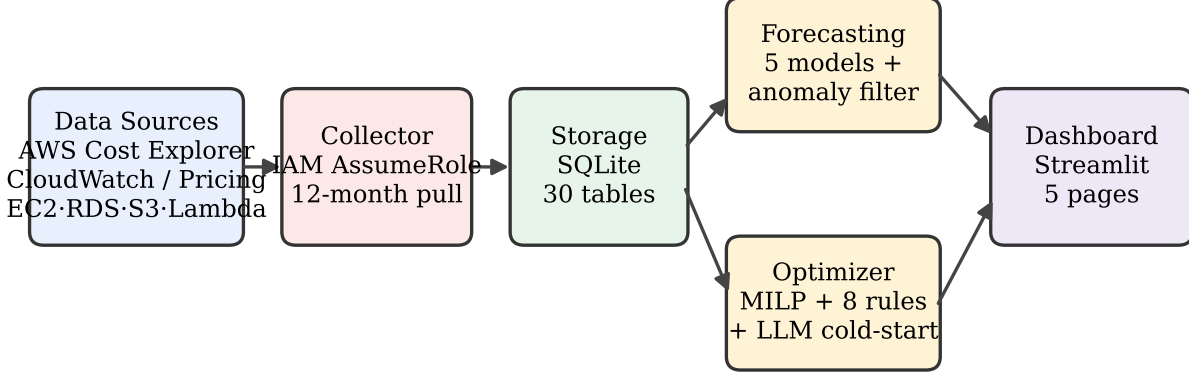


Fig. 1. Three-stage architecture. The collector populates a single store from which the forecasting (analyze) and optimization engines derive base capacity, surge headroom, and commitment recommendations.

TABLE I
NOTATION

Symbol	Meaning
d_t	realized demand (daily cost) on day t
b_t, s_t, u_t	base, seasonal, surge components
F, q^*	demand CDF and newsvendor reservation level
c_r, c_o	reserved / on-demand unit price
c_u, c_o^+	underage / overage cost
$x_{i,c}$	assign config. c to resource i (binary)
$\pi_{i,c}$	effective price (reserved if eligible)
$\rho_i^{\text{cpu}}, \rho_i^{\text{mem}}$	required vCPU / memory (sized)
γ	surge headroom factor (1.3)
(s, S)	supplementary-capacity thresholds

S ; expirations and cancellations are governed by symmetric thresholds [3], [8]. Because the firm does not know F and operates over a heterogeneous fleet, we realize this policy with a forecast-driven heuristic composed of three mechanisms.

A. Demand Forecasting and Surge Detection

The analyze stage estimates $b_t + s_t$ (the reservable base) and the surge tail u_t that determines headroom. We use an ensemble of five models behind a common `fit/predict` interface: *Naive*, *Seasonal Naive*, *ETS* (Holt-Winters, additive weekly seasonality) [13], *Prophet* [12], and auto-tuned *SARIMA* [20], each emitting a point forecast and a 95% interval that empirically approximates F . Before fitting, surges are removed so they do not bias the base estimate: day t is flagged if it deviates from its trailing 30-day window by more than three standard deviations *or* falls outside the interquartile fence $[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$, the union of two detectors [21]. Flagged points are excluded from training but retained to characterize the headroom the policy must provision. Models are selected by expanding-window (walk-forward) cross-validation reporting MAPE, RMSE, and MAE.

B. Right-Sizing as a Mixed-Integer Program

Given the estimated demand, right-sizing selects the cheapest surge-feasible configuration per resource. For a fleet $\mathcal{I}$ with candidate catalog $\mathcal{C}_i$, monthly hours H , and binary assignment $x_{i,c} \in \{0, 1\}$,

$$\min_x \sum_{i \in \mathcal{I}} \sum_{c \in \mathcal{C}_i} H \pi_{i,c} x_{i,c} \quad (3)$$

$$\text{s.t.} \sum_c x_{i,c} = 1, \quad \forall i; \quad \sum_c \text{vcpu}_c x_{i,c} \geq \rho_i^{\text{cpu}}, \quad \forall i; \quad (4)$$

$$\sum_c \text{mem}_c x_{i,c} \geq \rho_i^{\text{mem}}, \quad \forall i; \quad \sum_i \sum_c H \pi_{i,c} x_{i,c} \leq B, \quad (5)$$

where $\pi_{i,c} = c_r$ if resource i qualifies for a reservation (steady, long-running) and c_o otherwise, and B is an optional budget. The headroom encodes the supplementary capacity implied by the newsvendor fractile: rather than the average (which hides spikes) or the maximum (an outlier), we size to the 95th percentile of utilization scaled by γ ,

$$\rho_i^{\text{cpu}} = \frac{P_{95}(\text{CPU}_i)}{100} \cdot \text{vcpu}_i^{\text{cur}} \cdot \gamma, \quad \gamma = 1.3. \quad (6)$$

The factor $\gamma = 1.3$ reserves 30% as surge headroom—the data-driven counterpart of a supplementary contract sized to absorb u_t without paying for it year-round. Program (3)–(5) is solved per service with the CBC solver via PuLP [22]; a change is recommended only when the optimum is cheaper than the incumbent, and infeasible instances are reported and skipped.

C. Commitment Rules as a Threshold Policy

The (s, S) thresholds are instantiated by eight rules (Table II). The reservation rule is the policy’s core: a resource that has run steadily for at least 60 days—the lower threshold s on sustained utilization—is promoted from on-demand to a one-year base contract, while right-sizing the configuration down

TABLE II
COMMITMENT RULES REALIZING THE (s, S) POLICY

Service	Trigger (threshold)	Action
EC2/RDS	on-demand, steady ≥ 60 d	reserve (base contract)
Lambda	avg mem $< 50\%$ alloc	downsize tier
EBS	gp2 / unattached / idle	gp3, delete
S3	STANDARD, < 100 req/day	intelligent-tiering
DynamoDB	provisioned, util $< 50\%$	on-demand mode
NAT	cost $> \$30$ /mo	VPC endpoint
ELB	no targets, no traffic	delete idle

plays the role of cancelling or letting supplementary capacity expire once load falls below the upper threshold S . The remaining rules cover services for which an integer program is unnecessary. A final deduplication keeps, per resource, the single highest-saving recommendation. For new tenants with no telemetry, an LLM-assisted cold-start module (Gemini 2.5 Flash [23]) maps a short questionnaire on expected traffic pattern and availability to an initial architecture, addressing the cold-start case of reservation planning.

V. NUMERICAL STUDY

A. Dataset and Setup

Mirroring the simulation-based evaluation of the anchor paper, we study a twelve-month workload (365 daily-cost observations, July 2024–June 2025) for a mid-size SaaS tenant. The workload is built from open production traces—the Bitbrains GWA-T-12 VM utilization dataset [24] and the Numenta Anomaly Benchmark [21]—combined with live AWS Pricing API data, so utilization shapes, anomalies, and prices are realistic. The tenant’s mean cost is \$74.64/day (min \$36.10, max \$232.52), about \$2,239/month. Forecasting uses walk-forward cross-validation; optimization uses (3)–(6) with $\gamma = 1.3$. All figures and numbers are reproduced by `paper/make_figures.py` against the shipped database.

B. Surge Characterization

Fig. 2 shows the daily-cost signal with surges flagged by the rolling detector. A stable base around the median of \$70/day is punctuated by sharp, short-lived spikes reaching \$232/day—the intermittent random surges that motivate the base/supplementary split. The detector isolates these spikes cleanly, leaving a base series suitable for newsvendor reservation sizing.

C. Demand Estimation Accuracy

Table III and Fig. 3 report MAPE by horizon from walk-forward cross-validation—the accuracy with which the system estimates the distribution F feeding (2). Prophet is most accurate at short horizons (MAPE 7.9% at 7 days, 9.8% at 14 days), ETS dominates the medium range (10–90 days), and the seasonal baseline becomes competitive only at very long horizons where all models degrade. Because reservation decisions hinge on near-term base demand, the short-horizon

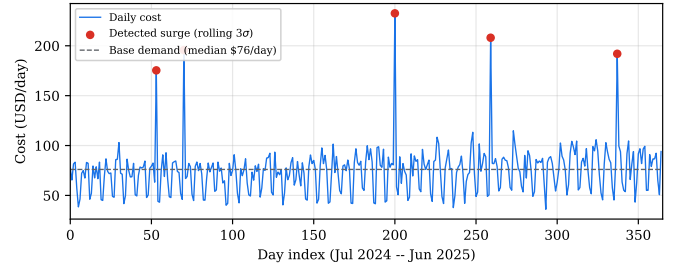


Fig. 2. Daily cost with detected demand surges. The base demand is served by reserved capacity; the surges define the supplementary headroom.

TABLE III
DEMAND-ESTIMATION MAPE (%) BY HORIZON (WALK-FORWARD CV)

Horizon (d)	Naive	Seas. Naive	ETS	Prophet
7	40.5	16.0	9.2	7.9
14	22.9	17.9	10.5	9.8
30	25.7	14.6	10.4	9.5
60	25.5	14.9	10.8	22.2
90	25.8	13.1	12.0	12.4
180	43.1	16.0	12.9	27.9
300	32.2	16.8	24.0	34.1

regime is operative, and there the ensemble achieves single-digit error. Fig. 4 shows a representative 30-day holdout in which the deployed forecaster tracks the base at 11.9% MAPE while the 95% interval—the empirical F —brackets the surges.

D. Cost Reduction

Applying the reservation and right-sizing policy to the tenant yields 19 recommendations totaling **\$590.40/month**, about 26% of the \$2,239 bill. Fig. 5 and Table IV break the savings down by service and action. The two dominant levers are exactly those predicted by the base/surge model: *right-sizing* surge headroom (RDS, \$362.81/month combined; a production PostgreSQL instance running far below its provisioned size is reduced, saving 93.3% on that resource) and *base-contract reservation* (EC2, \$208.42/month from promoting six steady, long-running instances from on-demand to one-year reserved pricing at $\approx 37\%$ discount, consistent with the critical fractile of (2)). Smaller, safe wins—deleting unattached storage, upgrading gp2 to gp3, replacing costly NAT paths with VPC endpoints, tiering cold objects, and downsizing an over-allocated function—account for the remainder. Each recommendation carries a confidence label and a human-readable rationale, so an operator can audit it before applying.

E. Discussion and Threats to Validity

The study confirms the central hypothesis: serving forecasted base demand with reserved commitments while holding percentile-based headroom for surges recovers roughly a quarter of the bill without risking surge service levels, and the dominant savings come precisely from the two levers of the newsvendor/threshold model. Two threats to validity follow

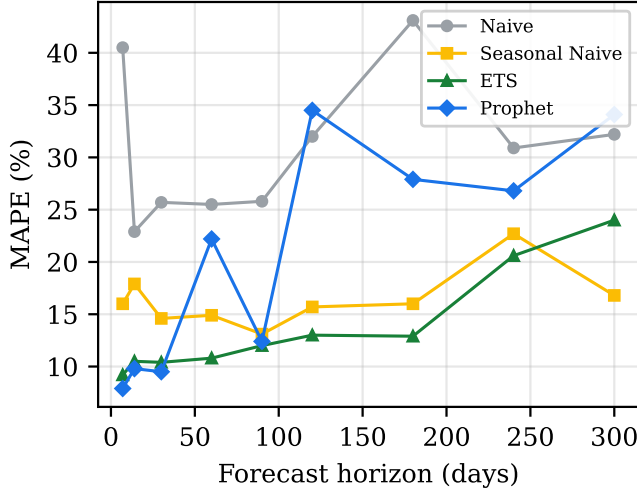


Fig. 3. Estimation error versus horizon. Prophet leads at short horizons; ETS is most robust through the mid-range relevant to reservation planning.

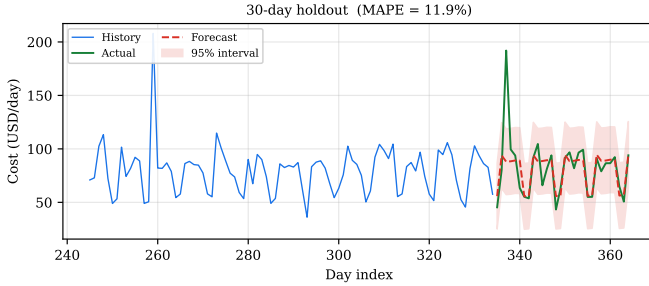


Fig. 4. A 30-day holdout forecast of daily cost. The forecast tracks base demand at 11.9% MAPE and the 95% interval brackets the surges.

those of the anchor work. First, like the anchor paper’s numerical study, our evaluation uses a workload synthesized from open traces rather than a live production account; absolute savings will vary by tenant. Second, accuracy is measured on a single tenant’s cost signal; broader validation is future work. Neither affects the mechanism, which is account- and dataset-agnostic.

VI. CONCLUSION AND FUTURE WORK

We presented Smart Cloud Optimizer, a forecast-driven realization of the capacity-reservation theory for intermittent random demand surges [3]. By decomposing demand into a reservable base and a surge component, estimating the demand distribution with an anomaly-aware ensemble, setting reserved capacity from the newsvendor critical fractile, and realizing the (s, S) supplementary-capacity policy through a mixed-integer program and commitment rules, the system recovers \$590.40/month ($\approx 26\%$) on a realistic twelve-month workload while estimating near-term demand at single-digit MAPE. Consistent with the open problems identified for this line of work, future work includes adding a third *spot*/preemptible tier to the reservation portfolio, modelling \$/hour Savings-

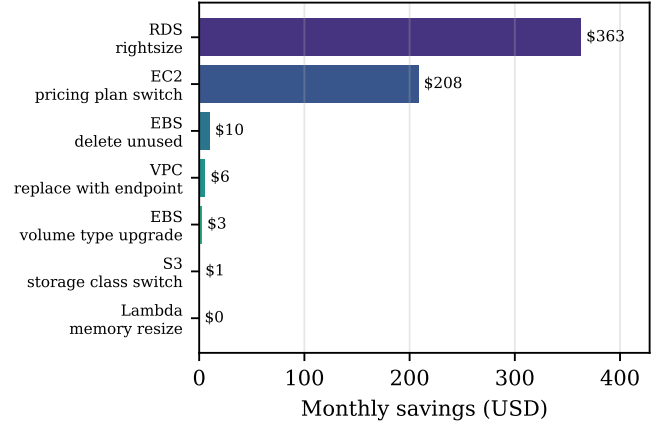


Fig. 5. Monthly savings by service and action. Base-contract reservation (EC2) and surge-headroom right-sizing (RDS) dominate, as the model predicts.

TABLE IV
RECOMMENDATION BREAKDOWN (MONTHLY SAVINGS)

Service	Action	Count	Savings (\$)
RDS	right-size (headroom)	2	362.81
EC2	reserve (base contract)	6	208.42
EBS	delete unused	3	10.00
VPC	replace with endpoint	2	5.67
EBS	gp2 $\rightarrow$ gp3	4	2.60
S3	storage-class switch	1	0.79
Lambda	memory resize	1	0.11
Total		19	590.40

Plan commitments in place of instance-count reservations, handling non-stationary (regime-switching) demand with on-line change-point detection, and validating on live multi-tenant accounts with automatic application and monitoring of the recommended actions.

ACKNOWLEDGMENT

The authors thank the Faculty of Computer and Information Sciences, Ain Shams University, and acknowledge the providers of the Bitbrains GWA-T-12 and Numenta Anomaly Benchmark datasets.

REFERENCES

- [1] Gartner, Inc., “Gartner forecasts worldwide public cloud end-user spending to reach nearly \$600 billion,” *Gartner Press Release*, 2023.
- [2] Flexera, “State of the cloud report,” *Flexera*, 2024.
- [3] S. Chen, J. Lei, and K. Moinszadeh, “Cost optimization in cloud computing: Capacity reservation for intermittent random demand surges,” *Production and Operations Management*, vol. 33, no. 6, pp. 1265–1284, 2024.
- [4] Amazon Web Services, “Amazon EC2 pricing and reserved instances,” <https://aws.amazon.com/ec2/pricing/>, 2024.
- [5] S. Chaisiri, B.-S. Lee, and D. Niyato, “Optimization of resource provisioning cost in cloud computing,” *IEEE Transactions on Services Computing*, vol. 5, no. 2, pp. 164–177, 2012.
- [6] M. Hadji, B. Aupetit, and D. Zeghlache, “Multi-stage stochastic programming for cloud resource provisioning,” *European Journal of Operational Research*, vol. 292, no. 1, pp. 291–306, 2021.

- [7] D. Bertsimas and M. Sim, “The price of robustness,” *Operations Research*, vol. 52, no. 1, pp. 35–53, 2004.
- [8] E. L. Porteus, *Foundations of Stochastic Inventory Theory*. Stanford University Press, 2002.
- [9] E. Furman and A. Diamant, “Optimal capacity planning for cloud service providers with periodic, time-varying demand,” *European Journal of Operational Research*, vol. 322, no. 1, pp. 133–146, 2025.
- [10] G. Casale, N. Mi, L. Cherkasova, and E. Smirni, “Dealing with burstiness in multi-tier applications: Models and their parameterization,” *IEEE Transactions on Software Engineering*, vol. 38, no. 5, pp. 1040–1053, 2012.
- [11] R. N. Calheiros, E. Masoumi, R. Ranjan, and R. Buyya, “Workload prediction using ARIMA model and its impact on cloud applications’ qos,” *IEEE Transactions on Cloud Computing*, vol. 3, no. 4, pp. 449–458, 2015.
- [12] S. J. Taylor and B. Letham, “Forecasting at scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [13] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021.
- [14] S. Arbat, V. K. Jayakumar, J. Lee, W. Wang, and I. K. Kim, “Wasserstein adversarial transformer for cloud workload prediction,” in *Proc. AAAI Conf. Artificial Intelligence*, 2022.
- [15] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proc. 26th Symp. Operating Systems Principles (SOSP)*, 2017, pp. 153–167.
- [16] Y. Garí, D. A. Monge, E. Pacini, C. Mateos, and C. G. Garino, “Reinforcement learning-based application autoscaling in the cloud: A survey,” *Engineering Applications of Artificial Intelligence*, vol. 102, p. 104288, 2021.
- [17] T. Llorido-Botran, J. Miguel-Alonso, and J. A. Lozano, “A review of auto-scaling techniques for elastic applications in cloud environments,” *Journal of Grid Computing*, vol. 12, no. 4, pp. 559–592, 2014.
- [18] N. Roy, A. Dubey, and A. Gokhale, “Efficient autoscaling in the cloud using predictive models for workload forecasting,” in *Proc. IEEE 4th Int. Conf. Cloud Computing (CLOUD)*, 2011, pp. 500–507.
- [19] J. R. Stormont and M. Fuller, “Cloud FinOps: Collaborative, real-time cloud financial management,” *O’Reilly Media*, 2023.
- [20] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.
- [21] A. Lavin and S. Ahmad, “Evaluating real-time anomaly detection algorithms – the numenta anomaly benchmark,” in *Proc. 14th IEEE Int. Conf. Machine Learning and Applications (ICMLA)*, 2015, pp. 38–44.
- [22] S. Mitchell, M. O’Sullivan, and I. Dunning, “PuLP: A linear programming toolkit for Python,” *The University of Auckland*, 2011.
- [23] Google DeepMind, “Gemini 2.5: Multimodal models for reasoning and generation,” <https://deepmind.google/technologies/gemini/>, 2024.
- [24] S. Shen, V. van Beek, and A. Iosup, “Statistical characterization of business-critical workloads hosted in cloud datacenters,” in *Proc. 15th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Computing (CCGrid)*, 2015, pp. 465–474.